Day 7 – Final Code Cleanup, Architecture Diagram, and GitHub Preparation

1. Project Status Overview

Day 7 focused on preparing VisionGuard Phase 1 for professional presentation and future upload. After completing the working AI + Arduino hardware system in earlier phases, today's work emphasized final polishing, documentation readiness, project structure, and visual explanation of the system architecture.

The objective was to make the project easier to understand, easier to maintain, and ready for GitHub or academic submission.

---

2. Final Code Cleanup

The main VisionGuard code was reviewed and cleaned to improve readability and maintainability.

Cleanup Focus Areas

```
1. Removed unnecessary duplicate logic
2. Grouped related code sections together
3. Improved naming consistency
4. Organized imports properly
5. Ensured cleanup functions run safely at the end
6. Kept AI logic, Arduino logic, and UI logic clearly separated
```

Cleaned Code Structure

```
Import libraries
↓
Initialize Arduino communication
↓
Define Arduino control function
↓
Create captures folder
↓
Initialize variables
↓
Load face detection model
↓
Start webcam
↓
Run main loop
↓
Motion detection
↓
Face detection
↓
```

```
UI display
↓
Arduino alert logic
↓
Draw visualization
↓
Cleanup resources
```

This structure makes the program easier to debug and easier to explain during viva, interviews, or project review.

---

3. Comments Added Where Needed

Comments were added to explain important logic blocks, especially where beginners may find the code difficult.

Commented Areas

```
Arduino serial communication
Motion detection pipeline
Gaussian blur usage
Thresholding and dilation
Motion memory logic
Face detection
State-based alert system
Cooldown-based image saving
OpenCV display and exit condition
Final cleanup
```

Example: Motion Detection Comments

```
# Convert previous and current frames to grayscale
# Grayscale reduces processing load and simplifies comparison
prev_gray = cv2.cvtColor(prev_frame, cv2.COLOR_BGR2GRAY)
curr_gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

# Apply Gaussian blur to reduce camera noise and small lighting changes
prev_gray = cv2.GaussianBlur(prev_gray, (9, 9), 0)
curr_gray = cv2.GaussianBlur(curr_gray, (9, 9), 0)

# Compare previous and current frames to detect changes
diff = cv2.absdiff(prev_gray, curr_gray)

# Convert difference image into binary motion mask
_, thresh = cv2.threshold(diff, 40, 255, cv2.THRESH_BINARY)
```

Example: Arduino Control Comment

```
def send_to_arduino(signal):
    # Send '1' to Arduino to turn ON LED and buzzer
    # Send '0' to Arduino to turn OFF LED and buzzer
    if signal == 1:
        arduino.write(b'1')
        print("Arduino Alert ON")
    else:
        arduino.write(b'0')
        print("Arduino Alert OFF")
```

The comments improve the educational value of the code and help future revision.

4. Final Architecture Diagram Created

A final Phase 1 architecture diagram was created to visually explain the complete VisionGuard system.

Diagram Title

```
VisionGuard - Phase 1 Final Architecture
```

Diagram Subtitle

```
AI Intruder Detection System with Arduino Uno Hardware Alert
```

Diagram Includes

```
Webcam / laptop camera
Video frame capture using Python + OpenCV
AI detection engine
Motion detection
Face detection
Motion memory
Decision logic
Evidence capture
PySerial communication
Arduino Uno R3
LED alert
Active buzzer alert
On-screen UI/status display
System behavior summary
```

Architecture Flow

```
Webcam / Laptop Camera
→ Video Frame Capture
→ AI Detection Engine
   → Motion Detection
   → Face Detection
   → Motion Memory
   → Decision Logic
→ Evidence Capture
→ Serial Communication
```

```
→ Arduino Uno R3
→ LED Alert + Buzzer Alert
```

Purpose of the Diagram

The architecture diagram helps reviewers quickly understand how the software AI layer communicates with the embedded hardware layer. It also shows that VisionGuard is not only a Python project, but a complete AI + Electronics system.

5. GitHub Upload Structure Prepared

The project folder was prepared in a clean GitHub-ready format.

Final GitHub Folder Structure

```
VisionGuard/
│
├── code/
│   ├── visionguard_main.py
│   └── arduino_test.py
│
├── arduino/
│   ├── command_mode.ino
│   └── led_buzzer_test.ino
│
├── docs/
│   ├── technical_reports/
│   │   ├── Day_1_Report.docx
│   │   ├── Day_2_Report.docx
│   │   ├── Day_3_Report.docx
│   │   ├── Day_4_Report.docx
│   │   ├── Day_5_Report.docx
│   │   ├── Day_6_Report.docx
│   │   └── Day_7_Report.docx
│   │
│   └── architecture/
│       └── phase_1_architecture.png
│
├── images/
│   ├── prototype_circuit.jpg
│   ├── complete_demo.jpg
│   ├── tinkercad_circuit.png
│   └── wiring_diagram.png
│
├── captures/
│   └── sample_intruder_capture.jpg
│
├── requirements.txt
├── README.md
├── .gitignore
└── LICENSE
```

This structure separates source code, Arduino code, documentation, images, captured samples, and setup files.

---

6. Supporting Files Prepared

requirements.txt

A `requirements.txt` file was planned for Python dependencies.

```
opencv-python
pyserial
numpy
```

This allows anyone to install required libraries easily using:

```
pip install -r requirements.txt
```

.gitignore

A `.gitignore` file was planned to keep unnecessary files out of GitHub.

```
__pycache__/
*.pyc
.venv/
env/
captures/intruder_*.jpg
.DS_Store
```

This prevents temporary files, virtual environments, and multiple captured intruder images from being uploaded.

README.md

The README file was prepared to include:

```
Project title
Short description
Features
Hardware used
Software used
System workflow
Folder structure
How to run
Arduino setup
Future scope
```

The README will act as the first explanation page for anyone viewing the project.

7. Next Development Roadmap

The next phase will move VisionGuard toward IoT capability using ESP32.

```
Phase 2: ESP32 IoT Alert System
1. Connect ESP32 with LED and buzzer
2. Test ESP32 serial/Wi-Fi control
3. Add Telegram or email alert
4. Build remote notification workflow
```